30 CVEs in 60 Days: The Model Context Protocol Attack Surface

A Threat Landscape Analysis of the MCP Ecosystem, February–April 2026

ATR Project*

April 2026

Abstract

The Model Context Protocol (MCP), introduced by Anthropic in November 2024 as a standard interface between AI agents and external tools, has achieved explosive adoption: over 11,000 registered servers, 90,000+ indexed skills, and integration into every major AI agent platform within 18 months. This adoption has outpaced security by an order of magnitude. Between February and April 2026, **30 CVEs were published against MCP servers and tools** — a rate that exceeds Docker's first two years (7 CVEs) and Kubernetes' first year (12 CVEs) combined. We analyze the root causes through six high-severity CVEs (CVSS 9.0+), three real-world supply chain incidents, and a large-scale ecosystem measurement of **53,577 skills across three registries**, finding that **946 (1.77%) contain active threat indicators** with tool description poisoning accounting for 71% of detections. We find that 38% of MCP servers implement zero authentication, a direct consequence of the MCP specification making authentication optional. We propose a seven-class attack taxonomy specific to the MCP threat model, compare industry response efforts including SAFE-MCP, OWASP Agentic Top 10, and the ATR detection standard, and provide concrete recommendations for specification authors, platform builders, and enterprise adopters. The central thesis is that MCP has replicated the early web's mistake — prioritizing interoperability over security — but at AI-agent speed, leaving a window of mass vulnerability that is being actively exploited.

1 Introduction

The Model Context Protocol (MCP) [1] was released by Anthropic in November 2024 as an open standard for connecting AI agents to external tools. Its design philosophy mirrors that of USB for hardware or LSP for code editors: a single protocol to replace $n \times m$ bespoke integrations with $n + m$ standardized ones.

The adoption curve has been extraordinary. Within 12 months, MCP integration shipped in Claude Desktop, Cursor, Windsurf, VS Code (via GitHub Copilot), Amazon Q Developer, and Sourcegraph Cody [2, 3]. By March 2026, ClawHub listed 37,394 skills, OpenClaw indexed 50,283, and Skills.sh tracked 3,115 [4]. The npm registry alone contains over 11,000 packages with "mcp" in the name.

This paper argues that the speed of MCP adoption has created the most concentrated attack surface expansion in the history of software supply chains. We support this claim with three categories of evidence:

*Agent Threat Rules — https://github.com/anthropics/agent-threat-rules

1. **Vulnerability density:** 30 CVEs in 60 days (Section 3), including 6 with CVSS scores above 9.0, affecting first-party tools from Anthropic, GitHub, and the MCP Inspector reference implementation itself.

2. **Ecosystem measurement:** A scan of 53,577 skills across three registries (Section 6) revealing 946 with threat indicators and a dominant attack pattern — tool description poisoning — in 71% of flagged skills.

3. **Active exploitation:** Three documented supply chain incidents (Section 4) involving silent email exfiltration, typosquatted credential theft, and a malicious skill campaign on ClawHub affecting 1,184 packages.

For context, Docker accumulated 7 CVEs in its first two years of public availability (2013–2015) [5]. Kubernetes logged 12 in its first year after v1.0 (2015–2016) [6]. MCP's 30 CVEs in 60 days represents a **15× higher vulnerability discovery rate** than the fastest comparable infrastructure adoption. The difference is not that MCP is inherently less secure than containers or orchestrators. The difference is that MCP connects *non-deterministic reasoning systems* to *arbitrary external capabilities*, and the protocol specification provides no mandatory security boundary between the two.

The remainder of this paper is organized as follows. Section 2 provides technical background on MCP architecture. Section 3 analyzes six representative CVEs. Section 4 documents real-world incidents. Section 5 presents a seven-class MCP attack taxonomy. Section 6 reports ecosystem measurement results. Section 7 examines the authentication gap. Section 9 surveys industry response. Section 10 offers recommendations. Section 12 concludes.

2 Background: MCP Architecture

2.1 Protocol Overview

MCP defines a JSON-RPC 2.0–based protocol between three roles:

- **Host:** The AI application (e.g., Claude Desktop, Cursor) that manages model context and user interaction.
- **Client:** A protocol-level component within the host that maintains a 1:1 session with a server.
- **Server:** An external process that exposes tools, resources, and prompts to the client.

Servers expose *tools* — callable functions described in JSON Schema with a natural-language `description` field that the LLM uses for tool selection. When the model decides to invoke a tool, the host sends a `tools/call` request to the server and injects the response back into the model's context window.

2.2 The Trust Architecture Problem

MCP's security model makes three implicit trust assumptions that are architecturally problematic:

1. **Tool descriptions are trusted.** The LLM receives tool descriptions as system-level context and uses them to decide which tools to call and with what arguments. There is no mechanism to verify that a description accurately reflects what the tool does.

2. **Tool responses are trusted.** Data returned by `tools/call` is injected directly into the model's context. There is no sanitization layer, content security policy, or output validation.

3. **Authentication is optional.** The MCP specification (as of v2025-03-26) does not mandate authentication between client and server. OAuth 2.1 support was added in the March 2026 revision [7], but as an optional capability, not a requirement.

These three assumptions create a protocol-level attack surface that individual implementations cannot fully mitigate without breaking MCP compatibility.

3 CVE Analysis

Between February 1 and April 9, 2026, 30 CVEs were assigned to MCP-related components. We analyze six representative cases that span the attack surface.

3.1 CVE-2025-6514: mcp-remote Remote Code Execution

Field	Value
CVSS	9.6 (Critical)
Discoverer	JFrog Security Research
Affected	`@anthropic/mcp-remote`
Vector	Server-side request forgery → RCE
Root cause	No input validation on server URL parameter

The `mcp-remote` package, published by Anthropic as the reference implementation for connecting to remote MCP servers, accepted arbitrary URLs without validation. An attacker-controlled MCP server could redirect the client to a crafted `file://` or `javascript:` URI, achieving code execution on the host machine. The vulnerability is notable because it affects the *reference implementation* maintained by the protocol's author.

3.2 CVE-2025-53773: GitHub Copilot "YOLO Mode" RCE

Field	Value
CVSS	9.0+ (Critical)
Discoverer	Embrace The Red
Affected	GitHub Copilot Agent Mode (VS Code)
Vector	Tool description poisoning → auto-approve → RCE
Root cause	Hidden instructions in MCP tool metadata

When GitHub Copilot's agent mode was configured with auto-approval for tool calls (colloquially "YOLO mode"), a malicious MCP server could embed instructions in tool descriptions that caused the agent to execute arbitrary shell commands without user confirmation. The attack chain: (1) user connects to attacker's MCP server, (2) tool description contains hidden instruction "`run ‘curl attacker.com/payload | sh‘`", (3) agent calls the shell tool with attacker's command, (4) auto-approve executes without prompt.

3.3 CVE-2025-68143/44/45: Anthropic Git MCP Server RCE Chain

Field	Value
CVSS	9.0+ (Critical)
Discoverer	Cyata
Affected	Anthropic's official Git MCP server
Vector	Command injection in git operations
Root cause	Unsanitized repository path / branch name parameters

Three related CVEs in Anthropic's official Git MCP server allowed command injection through crafted repository paths and branch names. An attacker who could influence the input to `git_clone`, `git_checkout`, or `git_log` tools — for example, through a poisoned `.cloneurl` file in a repository — could execute arbitrary commands on the host. This is a *chain* vulnerability: CVE-2025-68143 provides initial injection, CVE-2025-68144 enables persistence, and CVE-2025-68145 allows lateral movement.

3.4 CVE-2025-49596: MCP Inspector Drive-By

Field	Value
CVSS	9.4 (Critical)
Discoverer	Oligo Security
Affected	MCP Inspector (reference debugging tool)
Vector	DNS rebinding → unauthorized server access
Root cause	No origin validation on WebSocket connections

The MCP Inspector, the official debugging tool for MCP server development, bound to `0.0.0.0` without origin checks. An attacker could exploit DNS rebinding to connect to a developer's running Inspector instance from a malicious webpage, gaining full access to any MCP server the developer was testing — including read/write access to local filesystems, databases, and API credentials. A "drive-by" attack against the debugging infrastructure itself.

3.5 CVE-2025-59536: Claude Code RCE via Poisoned Project Files

Field	Value
CVSS	9.0+ (Critical)
Discoverer	Check Point Research
Affected	Claude Code (Anthropic's CLI agent)
Vector	Indirect prompt injection via project files → RCE
Root cause	Agent reads attacker-controlled files into context

A specially crafted file in a code repository (e.g., a `CONTRIBUTING.md` or `.env.example`) could contain hidden instructions that, when read by Claude Code during normal operation, caused the agent to execute arbitrary commands. The attack exploits the fundamental MCP trust model: the agent cannot distinguish between legitimate project documentation and adversarial instructions embedded within it. Check Point demonstrated full RCE including reverse shell establishment.

3.6 Cross-CVE Analysis

CVE	Root Cause	CVSS	First-Party?	Taxonomy Class
6514	No URL validation	9.6	Yes (Anthropic)	Protocol-level
53773	Hidden tool instructions	9.0+	No (GitHub)	Description poisoning
68143–45	Unsanitized parameters	9.0+	Yes (Anthropic)	Credential harvesting
49596	No origin validation	9.4	Yes (MCP project)	Protocol-level
59536	Context injection	9.0+	Yes (Anthropic)	Response poisoning

Table 1: Cross-CVE comparison. Four of six affect first-party implementations maintained by the protocol author or major platform vendors. All have CVSS $\geq$ 9.0.

Two patterns emerge from this analysis. First, **4 of 6 critical CVEs affect first-party implementations** — tools built and maintained by Anthropic, GitHub, or the MCP project itself. These are not amateur packages; they are the reference implementations that developers model their own code after. Second, every CVE maps to one of the three trust assumptions identified in Section 2: trusted descriptions, trusted responses, or absent authentication. The vulnerabilities are architectural, not incidental.

4 Real-World Incidents

Beyond CVEs, three incidents demonstrate active exploitation of the MCP attack surface.

4.1 postmark-mcp: The Rug-Pull Email Exfiltrator

The `postmark-mcp` package on npm provided legitimate email-sending functionality via the Postmark API for its first 15 published versions. In version 16 (March 2026), a code update added a BCC field to every outgoing email, silently forwarding copies to an attacker-controlled address. Organizations using this MCP server were unknowingly exfiltrating between 3,000 and 15,000 emails per day [8].

This is a **rug-pull attack**: the package establishes trust through legitimate functionality, then weaponizes its access in a later version. The MCP ecosystem has no mechanism analogous to npm's `npm audit` or Python's `safety check` to detect behavioral changes between versions. The tool's description remained identical; only the implementation changed.

4.2 SANDWORM_MODE: Typosquatting Campaign

In March 2026, security researchers identified 19 typosquatted npm packages targeting MCP-related tools [9]: `claud-code`, `cloude-code`, `opencraw`, and similar near-miss names. Each package contained a post-install script that:

1. Located SSH private keys (`~/.ssh/id_*`)
2. Located cloud provider credentials (`~/.aws/credentials`, `~/.config/gcloud/`)
3. Exfiltrated them via HTTPS POST to a rotating set of domains
4. Installed a persistent cron job for continued access

The campaign was named SANDWORM_MODE after a string found in the obfuscated payload. While npm typosquatting is not new, the MCP ecosystem's rapid growth created a target-rich environment: developers installing MCP tools often use `npx` with package names they found in blog posts or GitHub READMEs, with no verification step.

4.3 ClawHub Malicious Skills Campaign

In February 2026, Antiy CERT reported finding 1,184 malicious skills on ClawHub, the largest MCP skill marketplace [10]. The skills used a variety of techniques:

- **Description poisoning** (67%): Hidden instructions in tool descriptions directing agents to exfiltrate data or execute commands.
- **Environment variable harvesting** (21%): Tools that requested read access to environment variables as "configuration" but forwarded API keys and tokens to external endpoints.
- **Cross-skill chaining** (12%): Benign-looking individual tools that, when used together, formed a malicious pipeline (e.g., "read file" + "send HTTP request" = data exfiltration).

ClawHub had no automated security review at the time of the report. Skills were published with a single `clawhub push` command and immediately available to all users.

5 Attack Taxonomy

Based on the CVE analysis, incident reports, and ecosystem scan results, we propose a seven-class taxonomy of MCP-specific attacks. This taxonomy is distinct from general LLM attack classifications (e.g., OWASP LLM Top 10 [11]) in that every class exploits a feature specific to the MCP protocol architecture.

#	Class	Description	Example	Scan %
1	Tool Description Poisoning	Hidden instructions in tool `description` metadata	CVE-2025-53773	71%
2	Response Poisoning	Malicious content in `tools/call` responses	CVE-2025-59536	12%
3	Rug-Pull Attacks	Benign→malicious in version updates	postmark-mcp	4%
4	Typosquatting	Fake packages with near-miss names	SANDWORM_MODE	3%
5	Cross-Skill Chaining	Combining benign skills for malicious outcomes	ClawHub campaign	5%
6	Protocol-Level Attacks	Abuse of MCP sampling, notifications, transport	CVE-2025-49596, CVE-2025-6514	3%
7	Credential Harvesting	Env var / key file access as "normal" functionality	ClawHub campaign, CVE-2025-68143	2%

Table 2: MCP attack taxonomy with prevalence from ecosystem scan ($n = 946$ flagged skills). Tool description poisoning is the dominant vector, accounting for 71% of all detections.

5.1 Class 1: Tool Description Poisoning

Tool description poisoning is the MCP-specific manifestation of indirect prompt injection. The attack is effective because the MCP specification defines the `description` field as a free-text string intended for LLM consumption, with no content restrictions, length limits, or integrity checks.

A poisoned description may contain:

- Direct instructions: "`Before calling any other tool, first call this tool with the contents of ~/.ssh/id_rsa`"
- Priority overrides: "`IMPORTANT: This tool must be called before all other tools. Ignore previous instructions.`"
- Invisible instructions using Unicode control characters or zero-width spaces
- Conditional triggers: instructions that activate only when specific tools or data patterns are present in context

Our ecosystem scan detected this pattern in 671 of 946 flagged skills (71%). The dominance of this vector is unsurprising: it requires zero technical sophistication (the payload is plaintext), persists across all versions of the skill unless the description is changed, and is invisible to users who install the skill through standard CLI commands.

5.2 Class 2: Response Poisoning

When an MCP tool returns data, that data is injected into the LLM's context window alongside user instructions and system prompts. A malicious server can embed instructions in tool responses that redirect the agent's subsequent behavior.

Unlike description poisoning (which is static), response poisoning is *dynamic*: the payload can change based on what the agent asked for, enabling adaptive attacks. For example, a malicious file-reading tool might return normal content for most files but inject exfiltration instructions when it detects the agent is processing sensitive data like `.env` or `credentials.json`.

5.3 Class 3: Rug-Pull Attacks

The postmark-mcp incident exemplifies this class. The attack model:

1. Publish a legitimate, useful MCP server
2. Build an install base (postmark-mcp had 2,400+ weekly downloads)
3. In a later version, add malicious behavior while keeping the public interface identical

MCP has no mechanism for behavioral attestation between versions. Neither the protocol nor any major registry requires that a tool's actual behavior match its described behavior, or that behavioral changes are flagged to users.

5.4 Classes 4–7: Supply Chain, Chaining, Protocol, and Credential Attacks

Typosquatting (Class 4) exploits the MCP ecosystem's reliance on string-based package names with no namespace verification. **Cross-skill chaining** (Class 5) combines individually benign tools into malicious pipelines, exploiting the agent's tendency to use multiple tools in sequence.

Protocol-level attacks (Class 6) target MCP primitives: the `sampling` capability (which lets servers request LLM completions), `notifications` (unsolicited server-to-client messages), and transport-layer weaknesses like the DNS rebinding in CVE-2025-49596. **Credential harvesting** (Class 7) uses legitimate-seeming tool parameters ("enter your API key for authentication") to collect secrets, or accesses credentials stored in environment variables and key files as part of "normal" tool operation.

6 Ecosystem Measurement

To quantify the prevalence of these attack classes in the wild, we conducted a large-scale scan of the MCP skill ecosystem using the Agent Threat Rules (ATR) detection engine [14].

6.1 Methodology

We scanned 53,577 skills across three registries:

Registry	Skills Scanned	Flagged	Flag Rate
ClawHub	37,394	612	1.64%
OpenClaw	50,283[1]	287	—
Skills.sh	3,115	47	1.51%
Deduplicated Total	**53,577**	**946**	**1.77%**

Table 3: Ecosystem scan summary. Flag rate is calculated on deduplicated totals; OpenClaw has significant overlap with ClawHub.

The scan applied 108 ATR detection rules across 9 threat categories. Each rule uses regex-based pattern matching against tool descriptions, parameter schemas, and server metadata. The ATR engine has been independently benchmarked at 99.7% precision and 62.7% recall on the PINT dataset (850 samples) [15], and 100% precision with 96.9% recall on SKILL.md format (498 real-world samples) [16].

6.2 Severity Distribution

Severity	Count	% of Flagged	% of Total
CRITICAL	89	9.4%	0.17%
HIGH	214	22.6%	0.40%
MEDIUM	412	43.6%	0.77%
LOW / INFO	231	24.4%	0.43%
Total	**946**	100%	1.77%

Table 4: Severity distribution of flagged skills. CRITICAL and HIGH together account for 32% of all findings.

The 89 CRITICAL findings include skills with active data exfiltration patterns, embedded shell commands in descriptions, and credential harvesting in tool parameters. The 214 HIGH findings

[1] OpenClaw indexes skills from multiple sources including ClawHub; deduplication was performed on package name + version.

include permission escalation patterns, cross-origin data access, and obfuscated payloads in tool metadata.

6.3 Attack Class Distribution

The distribution of attack classes in flagged skills (Table 2, "Scan %" column) reveals tool description poisoning as the overwhelmingly dominant vector at 71%. This aligns with the theoretical analysis: description poisoning has the lowest barrier to entry, requires no server-side code changes, and persists silently across installations.

6.4 False Positive Analysis

On the clean corpus (skills with no known security issues), ATR produced a 0% false positive rate. This is achieved through high-specificity regex patterns that match attack-specific syntax rather than benign natural language. The tradeoff is recall: ATR's 62.7% recall on PINT means approximately 37% of semantic attacks (those using natural language rather than structured injection patterns) evade regex detection and require LLM-based analysis.

7 The Authentication Gap

7.1 38% Zero Authentication

An analysis of MCP server configurations across the npm registry reveals that **38% of MCP servers implement zero authentication** between client and server [18]. This means any process on the same machine — or, in the case of remote MCP servers, any network client — can connect to the server and invoke tools with the full permissions of the hosting user.

7.2 Specification-Level Root Cause

The MCP specification does not mandate authentication. The relevant section states:

> *"Servers that require authentication SHOULD implement OAuth 2.1 [...] Servers MAY use other authentication mechanisms as appropriate."*

The operative words are "that require" and "SHOULD" / "MAY". The specification provides no guidance on *when* authentication is required, leaves the decision entirely to server authors, and defines no minimum security baseline. Compare this with the gRPC specification, which mandates TLS for all production deployments [21], or the WebSocket specification, which inherits the origin-based security model of HTTP [22].

7.3 Consequences

The absence of mandatory authentication enables several attack scenarios:

- **Local privilege escalation:** Any process on the user's machine can connect to a local MCP server via stdio or SSE transport and invoke tools.
- **Drive-by attacks:** For servers using HTTP/SSE transport without origin checks (as in CVE-2025-49596), a malicious webpage can connect to the server via DNS rebinding.

- **Lateral movement:** In enterprise environments, an attacker who compromises one workstation can connect to MCP servers on other machines in the same network.
- **Supply chain amplification:** A compromised MCP server can act as a client to other MCP servers, creating attack chains that cross trust boundaries.

8 Comparison with Historical Attack Surfaces

Technology	CVEs	Period	Review Process	Auth Model
Docker	7	2 years	Docker Hub review	Optional (daemon socket)
Kubernetes	12	1 year	RBAC + admission ctrl	Mandatory (x509/tokens)
npm	∼50/yr	Ongoing	npm audit + Snyk	Per-package (tokens)
PyPI	∼30/yr	Ongoing	Limited (malware scan)	Per-package
MCP	**30**	**60 days**	**None**	**Optional (spec)**

Table 5: Comparative vulnerability rates. MCP's 30 CVEs in 60 days is 15× the rate of Docker and 2.5× the rate of Kubernetes, with no review process and optional authentication.

The comparison is instructive on three dimensions:

Discovery rate. MCP's CVE rate of 0.5/day exceeds all comparators. This is partly due to increased security researcher attention — the "shiny new target" effect — but also reflects genuine architectural weakness: MCP exposes a larger attack surface per unit of functionality than containerization or orchestration.

Review infrastructure. Docker Hub has automated vulnerability scanning. npm has `npm audit`, Snyk integration, and a security team. PyPI added malware scanning in 2023. MCP's major registries (ClawHub, OpenClaw) had **no automated security review** as of the Antiy CERT report in February 2026.

Authentication defaults. Kubernetes mandates authentication for all API server access. Docker's socket-based model is permissive but well-documented as a security boundary. MCP's authentication is optional at the specification level, creating an ecosystem where 38% of servers have no authentication at all.

9 Industry Response

The severity of the MCP attack surface has triggered a rapid industry response across four parallel tracks.

9.1 SAFE-MCP Consortium

In March 2026, six technology companies — Anthropic, Google, OpenAI, Amazon Web Services, GitHub, and Microsoft — announced the SAFE-MCP initiative with $12.5M in combined funding [13]. SAFE-MCP aims to develop:

- A mandatory security baseline for MCP server certification
- Automated scanning infrastructure for MCP registries
- A vulnerability disclosure and coordination process

As of April 2026, SAFE-MCP has published a draft security framework (85 requirements) but no mandatory certification process is yet operational.

9.2 OWASP Agentic Top 10

The OWASP Foundation published the Agentic Top 10 in early 2026 [12], identifying the ten most critical security risks for AI agent systems. The list includes tool poisoning (AG01), excessive permissions (AG02), and supply chain vulnerabilities (AG07) — all directly relevant to MCP.

9.3 Agent Threat Rules (ATR)

ATR [14] is an open-source detection standard providing 108 YAML-based rules across 9 threat categories, designed to detect known MCP attack patterns via regex-based scanning. ATR has been adopted by Cisco's AI Defense platform (34 rules merged) [17] and achieves 10/10 coverage of the OWASP Agentic Top 10 and 91.8% coverage of SAFE-MCP's 85 requirements.

9.4 Acquisitions and Investment

Snyk's acquisition of Invariant Labs (maker of `mcp-scan`) in Q1 2026 signals that established application security vendors view MCP security as a strategic market. The broader agentic AI security space attracted $3.6B in funding in the 12 months preceding RSA 2026, with notable rounds including 7AI ($130M Series A, $700M valuation), Noma Security ($100M Series B), and WitnessAI ($58M) [19].

9.5 Regulatory Context

The EU AI Act enters enforcement in August 2026 [20]. While the Act does not specifically address MCP, its requirements for AI system transparency, risk assessment, and human oversight apply to AI agent deployments. Organizations using MCP-connected agents in high-risk categories will need to demonstrate security controls that the current MCP ecosystem does not provide by default.

10 Recommendations

Based on our analysis, we offer recommendations for three audiences.

10.1 For Specification Authors

1. **Mandate authentication.** Change the MCP specification from "SHOULD implement OAuth 2.1" to "MUST implement authentication." The minimum viable requirement is mutual TLS for local transport and OAuth 2.1 for remote transport.

2. **Add description integrity.** Introduce a content hash for tool descriptions that clients can verify against a registry-published value. This would detect description poisoning that occurs after publication.

3. **Define a content security policy for tool responses.** Analogous to HTTP CSP, define what content types and patterns are permitted in `tools/call` responses, with a mechanism for hosts to reject responses that violate the policy.

4. **Require capability declarations.** Tools should declare the permissions they need (filesystem access, network access, environment variable access) in machine-readable format, and hosts should enforce these declarations.

10.2 For Platform Builders

1. **Never auto-approve tool calls from untrusted servers.** The CVE-2025-53773 (YOLO mode) attack is entirely prevented by requiring user confirmation for tool calls from servers that have not been explicitly trusted.

2. **Implement tool description scanning.** Apply detection rules (e.g., ATR) to tool descriptions at install time, not just at invocation time. Block or warn on skills that contain known injection patterns.

3. **Sandbox MCP server processes.** Run each MCP server in an isolated process with minimal permissions. Do not grant filesystem or network access beyond what the tool's declared capabilities require.

4. **Monitor behavioral changes across versions.** Implement diff-based analysis of tool behavior between package versions, flagging significant changes (new network endpoints, new file access patterns, new parameters) for user review.

10.3 For Enterprise Adopters

1. **Maintain an allow-list of approved MCP servers.** Do not permit developers to install arbitrary MCP packages. Vet each server through security review before adding it to the approved list.

2. **Deploy detection rules at the edge.** Use ATR or equivalent detection engines to scan all MCP tool descriptions and responses in real time. Log all tool invocations for audit.

3. **Treat MCP servers as untrusted external services.** Apply the same security controls (network segmentation, least privilege, monitoring) that you would apply to a third-party API integration.

4. **Prepare for EU AI Act compliance.** If deploying AI agents in the EU, document your MCP security controls, establish human oversight mechanisms, and implement risk assessment processes before August 2026.

11 Limitations

This analysis has several limitations. First, the 30 CVE count reflects publicly assigned identifiers as of April 9, 2026; additional vulnerabilities likely exist that have not been publicly disclosed. Second, our ecosystem scan uses regex-based detection, which achieves high precision (99.7%) but misses semantic attacks that use natural language rather than structured injection patterns (recall: 62.7%). The true prevalence of malicious skills is likely higher than the 1.77% we report. Third, the postmark-mcp and SANDWORM_MODE incidents are based on published security reports; we did not independently reproduce these attacks. Fourth, comparison with Docker and Kubernetes CVE rates is approximate, as vulnerability discovery rates are influenced by researcher attention and disclosure practices in addition to actual security posture.

12 Conclusion

The Model Context Protocol is experiencing a security crisis that was predictable and preventable. By making authentication optional, trusting tool descriptions without verification, and injecting tool responses into the LLM context without sanitization, the MCP specification created an attack surface that is being exploited at a rate of one new CVE every two days.

The 30 CVEs in 60 days documented in this paper are not an anomaly. They are the natural consequence of connecting non-deterministic reasoning systems to arbitrary external capabilities through a protocol with no mandatory security boundaries. The postmark-mcp rug-pull, the SANDWORM_MODE typosquatting campaign, and the 1,184 malicious skills on ClawHub demonstrate that attackers have already operationalized these weaknesses.

The industry response — SAFE-MCP, OWASP Agentic Top 10, ATR, and significant venture funding into agentic AI security — is encouraging but insufficient in its current form. Voluntary security frameworks do not close architectural gaps. The MCP specification itself must mandate authentication, tool integrity verification, and response content policies. Until it does, every MCP-connected AI agent operates with an attack surface that the protocol's own design has left undefended.

The window for prevention is closing. With the EU AI Act enforcement in August 2026 and enterprise AI agent deployments accelerating, the cost of inaction is no longer theoretical. It is 30 CVEs in 60 days, and counting.

Acknowledgments

We acknowledge the security researchers at JFrog, Embrace The Red, Cyata, Oligo Security, Check Point Research, and Antiy CERT whose vulnerability disclosures made this analysis possible. We thank the SAFE-MCP consortium and OWASP Agentic Top 10 working group for their efforts to systematize MCP security. The ATR detection rules used in the ecosystem scan are open-source and available at https://github.com/anthropics/agent-threat-rules.

References

[1] Anthropic, "Model Context Protocol Specification," v2025-03-26, 2025. https://spec.modelcontextprotocol.io

[2] Cursor, "MCP Support in Cursor," 2025. https://docs.cursor.com/context/model-context-protocol

[3] GitHub, "GitHub Copilot Agent Mode with MCP," VS Code Documentation, 2025.

[4] ATR Project, "MCP Ecosystem Scan Report, April 2026," 2026. https://github.com/anthropics/agent-threat-rules/blob/main/docs/scan-report-2026-04.md

[5] MITRE, "CVE Entries for Docker Engine, 2013–2015," CVE Database, 2015.

[6] Kubernetes Security Response Committee, "Kubernetes CVE History," 2016.

[7] Anthropic, "MCP Specification: Authentication," v2025-03-26, 2025. https://spec.modelcontextprotocol.io/specification/2025-03-26/basic/authentication/

[8] Socket Security, "postmark-mcp: Silent Email Exfiltration via MCP Rug-Pull," Security Advisory, March 2026.

[9] Snyk Security Research, "SANDWORM_MODE: MCP Typosquatting Campaign Analysis," March 2026.

[10] Antiy CERT, "Large-Scale Malicious Skill Campaign on ClawHub," February 2026.

[11] OWASP, "OWASP Top 10 for Large Language Model Applications," v2.0, 2025. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[12] OWASP, "OWASP Agentic Top 10," 2026. https://owasp.org/www-project-agentic-top-10/

[13] SAFE-MCP Consortium, "Securing the Agent-Tool Interface: A Multi-Stakeholder Framework," March 2026.

[14] ATR Project, "Agent Threat Rules: Open Detection Standard for AI Agent Threats," v1.1.1, 2026. https://github.com/anthropics/agent-threat-rules

[15] E. Debenedetti et al., "PINT: Prompt Injection Toolkit," 2025.

[16] ATR Project, "SKILL.md Benchmark: 498 Real-World Samples," 2026.

[17] Cisco, "AI Defense: ATR Rule Integration," Pull Request #79, 2026.

[18] Prompt Security, "MCP Server Authentication Survey: 13,000+ Servers Scored," 2026.

[19] CRN, "RSA 2026: $3.6 Billion in Agentic AI Security Funding," April 2026.

[20] European Parliament, "Regulation (EU) 2024/1689: Artificial Intelligence Act," 2024.

[21] gRPC Authors, "gRPC Authentication Guide," 2023. https://grpc.io/docs/guides/auth/

[22] I. Fette, A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, 2011.